1 Chained Hashing

Problem 1.1. Suppose that s is a character string. Recall the ASCII encoding for characters where every character has a corresponding integer. For example, "a" $= 97$, "b" $= 98$, and so on[1]. Let H be a hash table of size 7 implemented using **chained hashing** whose keys are strings. Suppose H uses the hash function

$$h(S) = \left(\sum_{i=0}^{n-1} s[i]\right) \bmod 7$$

where $S = s[0]s[1]s[2]\cdots s[n-1]$ is a character string.
For example, if $S =$ "cow", then $s[0] =$ 'c', $s[1] =$ 'o', $s[2] =$ 'w' and (using the ASCII table)

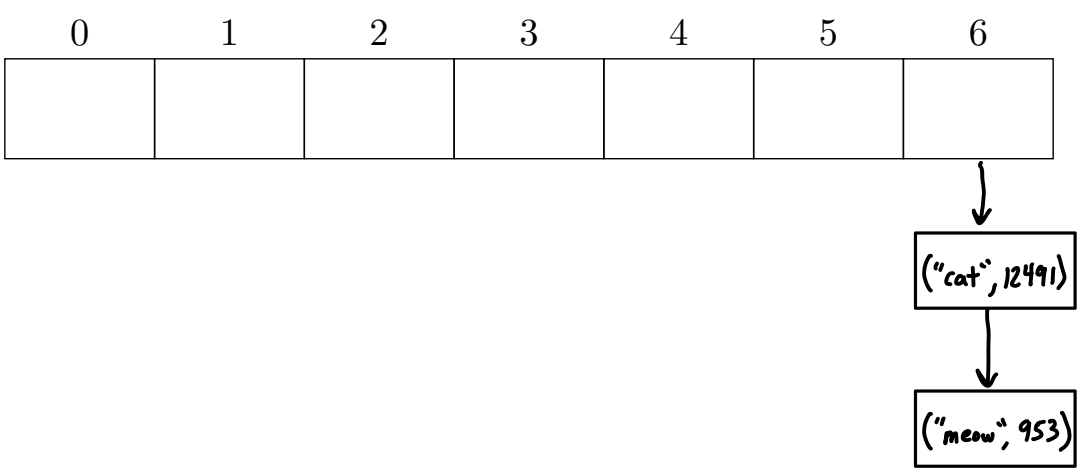
$$h(S) = \left(\sum_{i=0}^{2} s[i]\right) \bmod 7 = (99 + 111 + 119) \bmod 7 = (329) \bmod 7 = 0$$

Consider the following sequence of operations

`Insert`("cat", 12491)

`Insert`("meow", 953)

Draw the hash table after all of the above operations have been executed showing which elements are in which locations of the hash table and how they are stored. Feel free to use any software/calculator to help in computing the values!



$h(\text{"cat"}) = (99 + 97 + 116) \bmod 7 = 6$

$h(\text{"meow"}) = (109 + 101 + 111 + 119) \bmod 7 = 6$

[1] For specifics visit https://www.ascii-code.com/

Problem 1.2. For each of the following, explain why the hash function is a **BAD** choice.

(a) Let H be a hash table of size 128 with the hash function

$$h(K) = (22K) \bmod 128$$

implemented using **chained hashing**.

cuz $\gcd(22, 128) = 2$ means h only outputs even #s.

Thus, $\frac{1}{2}$ of table is used.

In general, if $\gcd \neq 1$, then only $1/\gcd$ locations used.

(b) Let H be a hash table of size 17 implemented using **chained hashing** whose keys are integers. Suppose H uses the hash function

$$h(K) = \left(\sum_{i=1}^{K^2} \left\lfloor \sqrt{i} \right\rfloor \right) \bmod 17$$

Does not take constant time to compute.
To get a numerical value, takes $\Theta(K^2)$ time.

(c) Let H be a hash table of size 50 with the hash function

$$h(K) = (3k + 7) \bmod 100$$

implemented using **chained hashing**.

$h(20) = 67 \bmod 100 = 67$ 67 is out of bounds!

Problem 1.3. Assume that H is a hash table implemented using chained hashing where the keys are integers and the data are floats. Write a kernel method that takes a float F as input and returns a list containing every key whose data is less than F.

This method should be written as a **kernel method**! Meaning you should not call other Hash Table functions during your implementation. Your code will be slow.

```
function HashTable.DataBelow(F)
    L = ∅
    foreach bucket in H
        foreach item in bucket
            if item.data < F
                Add item.key to L
            endif
        endfor
    endfor
    return L
```

Note: this takes $\Theta(n+m)$ time.

Searching DATA is SLOW.

2 Open Address Hashing

Problem 2.1. Let H be a hash table of size 11 with the hash function

$$h(K, j) = (3K + 2j + 1) \bmod 11$$

implemented using **open address hashing**. Any locations highlighted in **red** are occupied.

	0	1	2	3	4	5	6	7	8	9	10
H							$(1, D_1)$		$(7, D_7)$		

Consider the following sequence of operations

`Insert`$(1, D_1)$

`Insert`$(7, D_7)$

Draw the hash table after all of the above operations have been executed showing which elements are in which locations of the hash table and how they are stored. Show your work!

$h(1,0) = (3(1) + 2(0) + 1) \bmod 11 = 4$ ✗ j++

$h(1,1) = (3(1) + 2(1) + 1) \bmod 11 = 6$ ✓

$h(7,0) = (3(7) + 2(0) + 1) \bmod 11 = 0$ ✗ j++

$h(7,1) = (3(7) + 2(1) + 1) \bmod 11 = 2$ ✗ j++

$h(7,2) = (3(7) + 2(2) + 1) \bmod 11 = 4$ ✗ j++

$h(7,3) = (3(7) + 2(3) + 1) \bmod 11 = 6$ ✗ j++

$h(7,4) = (3(7) + 2(4) + 1) \bmod 11 = 8$ ✓

Problem 2.2. Let $A[]$ be an array of size n containing strings and let T be a string. We want to know if there are two entries $A[i]$ and $A[j]$ that when concatenated become T (with $i \neq j$). Your function should take $A[]$, n, and T as input and return **true** if there are two such values and **false** otherwise.

(a) Without using a hash table, implement an algorithm for determining whether any such values exist. Your algorithm should take no more than $O(n^2)$ time.

(b) Implement an alternative version of the the algorithm using a Hash Table. Your implementation should have an expected case running time of $O(n)$. Analyze your code and determine the **best**, **expected**, and **worst** case running time of your algorithm. Assume the size of the hash table is at least $2n$.

(c) Explain how this demonstrates that hash tables have actively made our code better.

Hint: This is very similar to Two Sum.

a)
```
function TwoString(A[], n, T)
    for i = 1 to n
        for j = 1 to n
            if (A[i] + A[j] = T OR A[j] + A[i] = T) AND i ≠ j
                return TRUE
            endif
        endfor
    endfor
    return FALSE
```

WC is in $\Theta(n^2)$, so always in $O(n^2)$.

b)

```
function TwoStrings(A[], n, T)
    H = new Hash Table
    for i = 1 to n
        if A[i] is a prefix of T
            S = T with A[i] removed from the front.
            if H.Member(S)
                return TRUE
            endif
        endif
        if A[i] is a suffix of T
            S = T with A[i] removed from the end.
            if H.Member(S)
                return TRUE
            endif
        endif
        if H.Member(A[i])
            H.Insert(A[i], "this doesn't matter")
        endif
    endfor
    return FALSE
```

python's splicing notation actually makes this easy!

```
m = A[i].length
if T[:m] = A[i]
    S = T[m:]
```

BC: Occurs when $A[1] + A[2] = T$. So, $B(n) \in \Theta(1)$.

EC: All methods take $\Theta(1)$ time.

Further, assuming no early return would be an upper bound.

So, $ET(n) \leq \sum_{i=1}^{n} c = cn$ Thus, $ET(n) \in O(n)$.

WC: All methods take $\Theta(s)$ time, where s = # of elements in Hash Table

Assume A contains no duplicates and no solution.

So, the body of the for loop takes $\Theta(s)$ time each iteration because it is a constant number of $\Theta(1)$ and $\Theta(s)$ operations.

SO, $W(n) = \sum_{i=1}^{n} cs$

$$W(n) = \sum_{i=1}^{n} ci = c\,\frac{n(n+1)}{2}$$

Thus, $W(n) \in \Theta(n^2)$.

Value of i	size (at end of loop)
1	1
2	2
3	3
4	4
⋮	⋮
n-1	n-1
n	n

c) $O(n)$ is **WAY** better. For example,

$n = 10000$ means the HT algorithm is **10000 TIMES FASTER.**

Some fun comments on running times!

1. Check if a string X is a prefix of Y is not really constant.
   ↳ really takes $\Theta(|X|)$, where $|X|$ is length of X.
2. Thus, all running times should be multiplied by w = max string length in A.
3. The real worst case running time would require some very particular inputs:
   1) $\forall i$, $A[i]$ is a prefix of T
   2) $\forall i$, $A[i]$ is a suffix of T
   3) No solution exists in A.

Problem 2.3 (Optional). Let $A[]$ be an array of integers of size of n and let T be an integer. We want to know if there are four entries $A[i]$, $A[j]$, $A[k]$, and $A[m]$ such that $A[i] + A[j] + A[k] + A[m] = T$ and i, j, k, m are all different. All of the algorithms should take $A[]$, n, and K as input and return **true** if there are two such values and **false** otherwise.

(a) Without using a hash table, implement an algorithm for determining whether there are such values or not. Determine the asymptotic complexity of your algorithm.

(b) Implement an alternative version of the algorithm using a Hash Table. Analyze your code and determine the best, expected, and worst case running time of your algorithm.

(c) Explain how this demonstrates that hashing have actually made our code better.

If you want an extra difficult coding challenge try to get the Hash Table implementation to have an expected case running time in $O(n^2)$.

optional